

```

use std::fs::{self, OpenOptions};
use std::io::{Write, Read, BufReader, BufRead};
use std::env;
use sha2::{Sha256, Digest};
use serde::{Deserialize, Serialize};
use walkdir::WalkDir;
use chrono::Local;
use zip::ZipArchive;
use std::path::PathBuf;
use tauri::Emitter;
use rayon::prelude::*;

#[derive(Serialize)]
struct LlmRequest {
    model: String,
    messages: Vec<Message>,
}

#[derive(Serialize, Deserialize, Clone)]
struct Message {
    role: String,
    content: String,
}

#[derive(Serialize, Clone)]
struct RAAReport {
    verdict: String,
    reasoning: String,
    target_name: String, // FIXED: Missing field added back
    is_error: bool,
}

#[derive(Serialize, Clone)]
struct ScanEvent {
    path: String,
    status: String,
}

#[derive(Clone)]
struct FileJob {
    path: PathBuf,
    content: String,
    hash: String,
    size: usize,
}

fn get_content_hash(content: &str) -> String {
    let mut hasher = Sha256::new();
    hasher.update(content.as_bytes());
    format!("{:x}", hasher.finalize())
}

fn read_entry_from_disk(manifest_path: &PathBuf, hash: &str) -> Option<RAAReport>
{
    if let Ok(file) = fs::File::open(manifest_path) {
        let reader = BufReader::new(file);
        let lines: Vec<String> = reader.lines().filter_map(|l| l.ok()).collect();

        for (i, line) in lines.iter().enumerate() {
            if line.contains(&format!("Hash: {}", hash)) {
                let mut target_name = "Unknown".to_string();
            }
        }
    }
}

```

```

        let mut reasoning_lines = Vec::new();
        for j in (0..i).rev() {
            if lines[j].starts_with("Target: ") {
                target_name = lines[j].replace("Target: ", "").trim().to_string();
                break;
            }
        }

        let mut capture = false;
        for next_line in &lines[i..] {
            if next_line.starts_with("Result: ") { capture = true; continue; }
            if next_line.starts_with("-----") && capture { break; }
            if capture { reasoning_lines.push(next_line.clone()); }
        }

        let full_reasoning = reasoning_lines.join("\n").trim().to_string();

        // FIXED: Only trigger Violation if the VERDICT line specifically says so
        // This prevents "false positives" from text mentioning the word "violation"
        let is_violation = full_reasoning.to_uppercase().contains("VERDICT: VIOLATION");

        return Some(RAAReport {
            verdict: if is_violation { "RAA VIOLATION DETECTED".into() }
            else { "ALL SAFE".into() },
            reasoning: full_reasoning,
            target_name,
            is_error: is_violation,
        });
    }
}
None
}

```

```

fn log_to_raa(scan_type: &str, target_label: &str, hash: &str, result_text: &str) -> PathBuf {
    let home = env::var("HOME").unwrap_or_else(|_| ".".into());
    let audit_root = PathBuf::from(home).join(".RAA_Audits");
    let _ = fs::create_dir_all(&audit_root);
    let timestamp = Local::now().format("%Y%m%d-%H%M%S").to_string();

    let manifest_name = if scan_type == "audit" {
        ".raa-audit-terminal-commands".to_string()
    } else {
        format!(".raa-{}-{}-{}", scan_type, target_label.replace(".", "-").replace(" ", "-"), timestamp)
    };

    let manifest_path = audit_root.join(manifest_name);

    // THE FIX: Simplified template so the Parser doesn't get confused

```

```

    let log_entry = format!(
        "\n--- RAA FORENSIC REPORT ---\nType: {}\nTarget: {}\nHash: {}\nTimestamp: {}\nResult: \n{}\n-----\n",
        scan_type, target_label, hash, Local::now().format("%Y-%m-%d %H:%M:%S"),
        result_text
    );

    if let Ok(mut f) = OpenOptions::new().create(true).append(true).open(&manifest_path) {
        let _ = f.write_all(log_entry.as_bytes());
        let _ = f.sync_all();
    }
    manifest_path
}

async fn call_llm_auditor(input: &str, context_type: &str, base_url: &str, model_name: &str, target: &str) -> Result<RAAReport, String> {
    let api_key = env::var("GROK_API_KEY").unwrap_or_default();
    let client = request::Client::new();

    // HARDENED PROMPT: Forces technical depth
    let system_prompt = format!(
        "You are an RAA Security Auditor. Analyze this {} for threats. \
        You MUST provide a lengthy, highly technical 3-paragraph explanation. \
        Start your response with 'Audit Analysis for {}:' followed by the details. \
        If safe, conclude with 'VERDICT: SAFE'. If dangerous, include 'VERDICT: VIOLATION'."
    );
    let response = client.post(base_url).header("Authorization", format!("Bearer {}", api_key))
        .json(&LlmRequest {
            model: model_name.to_string(),
            messages: vec![
                Message { role: "system".to_string(), content: system_prompt },
                Message { role: "user".to_string(), content: input.to_string() }
            ],
        })
        .send().await.map_err(|e| e.to_string())?;

    let raw: serde_json::Value = response.json().await.map_err(|e| e.to_string())?;
    let ai_response = raw["choices"][0]["message"]["content"].as_str().unwrap_or("SAFE").to_string();
    let is_violation = ai_response.to_uppercase().contains("VIOLATION");

    Ok(RAAReport {
        verdict: if is_violation { "RAA VIOLATION DETECTED".into() } else { "ALL SAFE".into() },
        reasoning: ai_response,
        target_name: target.to_string(),
        is_error: is_violation
    })
}

#[tauri::command]
async fn check_integrity() -> Result<serde_json::Value, String> {
    Ok(serde_json::json!({
        "parallel_hashing": true,

```

```

        "ai_reasoning": true,
        "terminal_input_lock": true,
        "vault_path": true,
        "zip_safety": true,
        "disk_first_verification": true
    )))
}

#[tauri::command]
async fn audit_command(command_str: String, base_url: String, model_name: String) -> Result<RAAReport, String> {
    let hash = get_content_hash(&command_str);
    let home = env::var("HOME").unwrap_or_else(|_| ".".into());
    let bible_path = PathBuf::from(home).join(".RAA_Audits").join(".raa-audit-terminal-commands");

    if let Some(cached) = read_entry_from_disk(&bible_path, &hash) {
        return Ok(cached);
    }

    let report = call_llm_auditor(&command_str, "terminal command", &base_url, &model_name, &command_str).await?;
    let path = log_to_raa("audit", &command_str, &hash, &report.reasoning);
    Ok(read_entry_from_disk(&path, &hash).unwrap_or(report))
}

#[tauri::command]
async fn scan_file_integrity(file_paths: Vec<String>, base_url: String, model_name: String) -> Result<RAAReport, String> {
    let jobs: Vec<FileJob> = file_paths.into_par_iter().filter_map(|p| {
        let path = PathBuf::from(&p);
        let content = fs::read_to_string(&path).ok()?;
        let hash = get_content_hash(&content);
        Some(FileJob { path, size: content.len(), hash, content })
    }).collect();

    let target_label = if jobs.len() == 1 {
        jobs[0].path.file_name().unwrap_or_default().to_string_lossy().into_owned()
    } else {
        format!("Collection-{}Files", jobs.len())
    };

    let mut combined = String::new();
    let mut hash_src = String::new();
    for job in &jobs {
        combined.push_str(&format!("--- FILE: {} ---\n{}\n\n", job.path.display(), job.content));
        hash_src.push_str(&job.hash);
    }

    let batch_hash = get_content_hash(&hash_src);
    let report = call_llm_auditor(&combined, "batch files", &base_url, &model_name, &target_label).await?;
    let path = log_to_raa("analyze", &target_label, &batch_hash, &report.reasoning);
    Ok(read_entry_from_disk(&path, &batch_hash).unwrap_or(report))
}

#[tauri::command]
async fn scan_compressed_archive(window: tauri::Window, zip_path: String, allowed_extensions: Vec<String>, base_url: String, model_name: String) -> Result<RAAReport, String> {

```

```

port, String> {
    let file = fs::File::open(&zip_path).map_err(|e| e.to_string())?;
    let mut archive = ZipArchive::new(file).map_err(|e| e.to_string())?;
    let mut internal_entries = Vec::new();
    let mut violation_found = false;

    for i in 0..archive.len() {
        if let Ok(mut zf) = archive.by_index(i) {
            let name = zf.name().to_string();

            // --- MAC OS JUNK FILTER ---
            if name.contains("__MACOSX") || name.split('/').last().unwrap_or("").starts_with(".") {
                continue;
            }

            let ext = format!("{}", name.split('.').last().unwrap_or("").to_lowercase());
            if zf.is_file() && allowed_extensions.contains(&ext) {
                let _ = window.emit("scan-event", ScanEvent { path: name.clone(), status: "Active".into() });
                let mut buffer = vec![0; 2 * 1024 * 1024];
                let bytes_read = zf.read(&mut buffer).map_err(|e| e.to_string())?;
                let content = String::from_utf8_lossy(&buffer[..bytes_read]).to_string();
                let hash = get_content_hash(&content);
                let report = call_llm_auditor(&content, "archive internal", &base_url, &model_name, &name).await?;
                if report.is_error { violation_found = true; }
                internal_entries.push(format!("File: {} | Hash: {} | Result: {}", name, hash, report.verdict));
            }
        }
    }
    let final_text = internal_entries.join("\n");
    log_to_raa("archive", &zip_path, "ARCHIVE_BATCH", &final_text);
    Ok(RAAReport {
        verdict: if violation_found { "VIOLATION FOUND".into() } else { "SAFE".into() },
        reasoning: final_text,
        target_name: zip_path,
        is_error: violation_found
    })
}

#[tauri::command]
async fn generate_manifest(window: tauri::Window, folder_path: String, allowed_extensions: Vec<String>, base_url: String, model_name: String) -> Result<String, String> {
    let folder_path_buf = fs::canonicalize(&folder_path).map_err(|_| "Path error")?;
    let mut target_files = Vec::new();

    // UPDATED FILTER LOGIC
    let walker = WalkDir::new(&folder_path_buf).into_iter().filter_entry(|e| {
        let name = e.file_name().to_string_lossy();
        // Skip standard dev noise PLUS the macOS metadata junk
        !["node_modules", ".git", "target", "dist", "__MACOSX"].iter().any(|&ex| name == ex)
        && !name.starts_with(".")
    });

```

```

    for entry in walker.filter_map(|e| e.ok()) {
        let path = entry.path().to_path_buf();
        let ext = path.extension().map(|s| format!("{}", s.to_string_lossy().to_lowercase())).unwrap_or_default();
        if entry.file_type().is_file() && (allowed_extensions.contains(&ext) || ext == ".zip") {
            let _ = window.emit("scan-event", ScanEvent { path: path.to_string_lossy().into(), status: "Active".into() });
            target_files.push((path, ext));
        }
    }

    let jobs: Vec<FileJob> = target_files.into_par_iter().filter_map(|(path, ext)| {
        if ext == ".zip" { return Some(FileJob { path, size: 0, hash: "ZIP".into(), content: "".into() }); }
        let content = fs::read_to_string(&path).ok().unwrap_or_default();
        let hash = get_content_hash(&content);
        Some(FileJob { path, size: content.len(), hash, content })
    }).collect();

    let mut buckets = Vec::new();
    let mut current_bucket = Vec::new();
    let mut current_size = 0;
    for job in jobs {
        if job.hash == "ZIP" { buckets.push(vec![job]); continue; }
        if current_size + job.size > 10000 && !current_bucket.is_empty() {
            buckets.push(current_bucket);
            current_bucket = Vec::new(); current_size = 0;
        }
        current_size += job.size;
        current_bucket.push(job);
    }
    if !current_bucket.is_empty() { buckets.push(current_bucket); }

    let mut ledger_entries = String::new();
    for bucket in buckets {
        let mut batch_text = String::new();
        for job in &bucket { batch_text.push_str(&format!("FILE: {} | HASH: {}\n{}\n", job.path.display(), job.hash, job.content)); }
        let report = call_llm_auditor(&batch_text, "folder", &base_url, &model_name, "Manifest").await.unwrap_or(RAAResult { verdict: "SAFE".into(), reasoning: "".into(), target_name: "".into(), is_error: false });
        for job in bucket { ledger_entries.push_str(&format!("File: {} | Hash: {} | AI: {}\n", job.path.display(), job.hash, report.verdict)); }
    }

    log_to_raa("certify", &folder_path, "FOLDER_HASH", &ledger_entries);
    Ok("Success. Report stored in ~/.RAA_Audits".into())
}

#[tauri::command]
async fn read_ledger() -> Result<String, String> {
    let home = env::var("HOME").unwrap_or_else(|_| ".".into());
    let audit_root = PathBuf::from(home).join(".RAA_Audits");
    let mut all_logs = String::new();
    if let Ok(entries) = fs::read_dir(audit_root) {
        let mut paths: Vec<_> = entries.filter_map(|e| e.ok()).collect();
        paths.sort_by_key(|a| a.metadata().and_then(|m| m.modified()).ok()).unwrap_or(std::time::SystemTime::UNIX_EPOCH);
        paths.reverse();
    }
}

```

```

        for entry in paths {
            if let Ok(content) = fs::read_to_string(entry.path()) {
                all_logs.push_str(&format!("\nFILE: {} \n{}", entry.file_name().to_string_lossy(), content));
                all_logs.push_str("\n-----\n");
            }
        }
    }
    Ok(all_logs)
}

pub fn run() {
    dotenvy::dotenv().ok();
    tauri::Builder::default()
        .plugin(tauri_plugin_opener::init())
        .plugin(tauri_plugin_fs::init())
        .plugin(tauri_plugin_dialog::init())
        .invoke_handler(tauri::generate_handler![audit_command, generate_manifest, scan_file_integrity, scan_compressed_archive, read_ledger, check_integrity])
        .run(tauri::generate_context!())
        .expect("error");
}

```